

Aula 5: Construtor e "Destructor" em Java

PROGRAMAÇÃO ORIENTADA A OBJETOS

Nesta aula, exploraremos um dos pilares fundamentais da orientação a objetos: os **construtores**. Aprenderemos como eles inicializam objetos, os tipos disponíveis, e como o Java gerencia o ciclo de vida e a destruição de objetos por meio do **Garbage Collection**.



Objetivos da Aula

1

Papel do Construtor

Compreender o papel fundamental do construtor na criação e inicialização de objetos em Java.

2

Tipos de Construtores

Aprender as diferenças entre construtores padrão, parametrizados e de cópia.

3

Ciclo de Vida

Entender as etapas que um objeto percorre desde sua criação até sua destruição.

4

Garbage Collection

Conhecer o mecanismo automático de gerenciamento de memória da JVM.

5

Prática

Aplicar construtores em exemplos reais com classes Java bem estruturadas.

O que é um Construtor?

Um **construtor** é um método especial em Java responsável por **inicializar** um objeto no momento de sua criação. Ele é chamado automaticamente quando utilizamos o operador `new`, configurando o estado inicial do objeto com os valores necessários para seu funcionamento.

Mesmo nome da classe

O construtor deve ter exatamente o mesmo nome da classe que pertence.

Sem tipo de retorno

Diferente de métodos comuns, o construtor não declara nenhum tipo de retorno — nem `void`.

Chamada automática

É invocado automaticamente pelo Java no momento em que o objeto é criado com `new`.



Java Jarvobiver
ioch {
 Object-construction EdgeCutter
 initialization();
}
 instlet-abfr = MandelJestrand();
 {
 forstructation();
 inichuder' = constructor();
 initialization();
 }
 just-Objecreniest-cojchiruste();
}

Constructor
constructor

Destructor
Destructor

Loid drode
phest ign binge

Características do Construtor

Regras Fundamentais

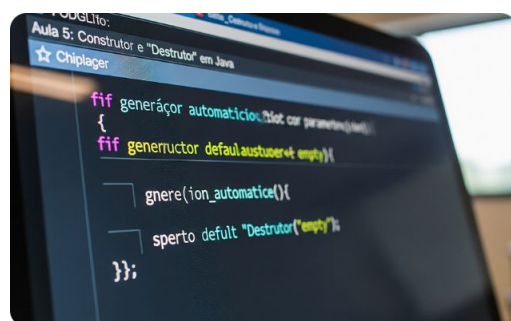
- 1 **Nome idêntico à classe:** o compilador Java identifica o construtor pelo nome que coincide com a classe.
- 2 **Sem tipo de retorno:** nem mesmo `void` deve ser declarado, caso contrário, o Java o tratará como método comum.
- 3 **Pode ter parâmetros:** aceita argumentos para personalizar a inicialização do objeto.
- 4 **Sobrecarga permitida:** é possível declarar múltiplos construtores com assinaturas diferentes na mesma classe.

Exemplo de Sintaxe

```
public class Pessoa {  
    String nome;  
    int idade;  
  
    // Construtor  
    public Pessoa(String nome,int idade) {  
        this.nome = nome;  
        this.idade = idade;  
    }  
}
```

- ❏ O construtor **não cria** o objeto — ele apenas o **inicializa**. A criação é feita pela JVM ao processar o operador `new`.

Tipos de Construtores em Java



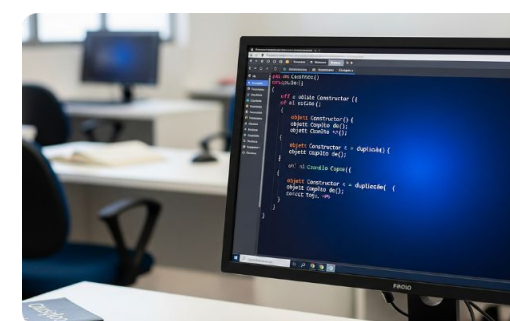
Construtor Padrão (Default)

Sem parâmetros, gerado **automaticamente** pelo Java quando nenhum outro construtor é declarado. Inicializa atributos com valores padrão (0, null, false).



Construtor Parametrizado

Recebe **argumentos** no momento da criação, permitindo inicializar o objeto com um estado definido. Essencial para garantir que os dados obrigatórios sejam fornecidos.



Construtor de Cópia

Não é nativo do Java, mas pode ser **implementado manualmente**. Recebe um objeto do mesmo tipo como argumento e copia seus atributos para o novo objeto.

Construtor Padrão (Default)

O Java fornece automaticamente um construtor padrão **sem parâmetros** quando nenhum construtor é declarado explicitamente na classe. Este construtor inicializa os atributos com seus valores padrão: `0` para números, `null` para objetos e `false` para booleanos.

📌 **Atenção:** Ao declarar qualquer construtor explícito, o construtor padrão **deixa de existir** automaticamente. É boa prática sempre defini-lo explicitamente quando necessário.

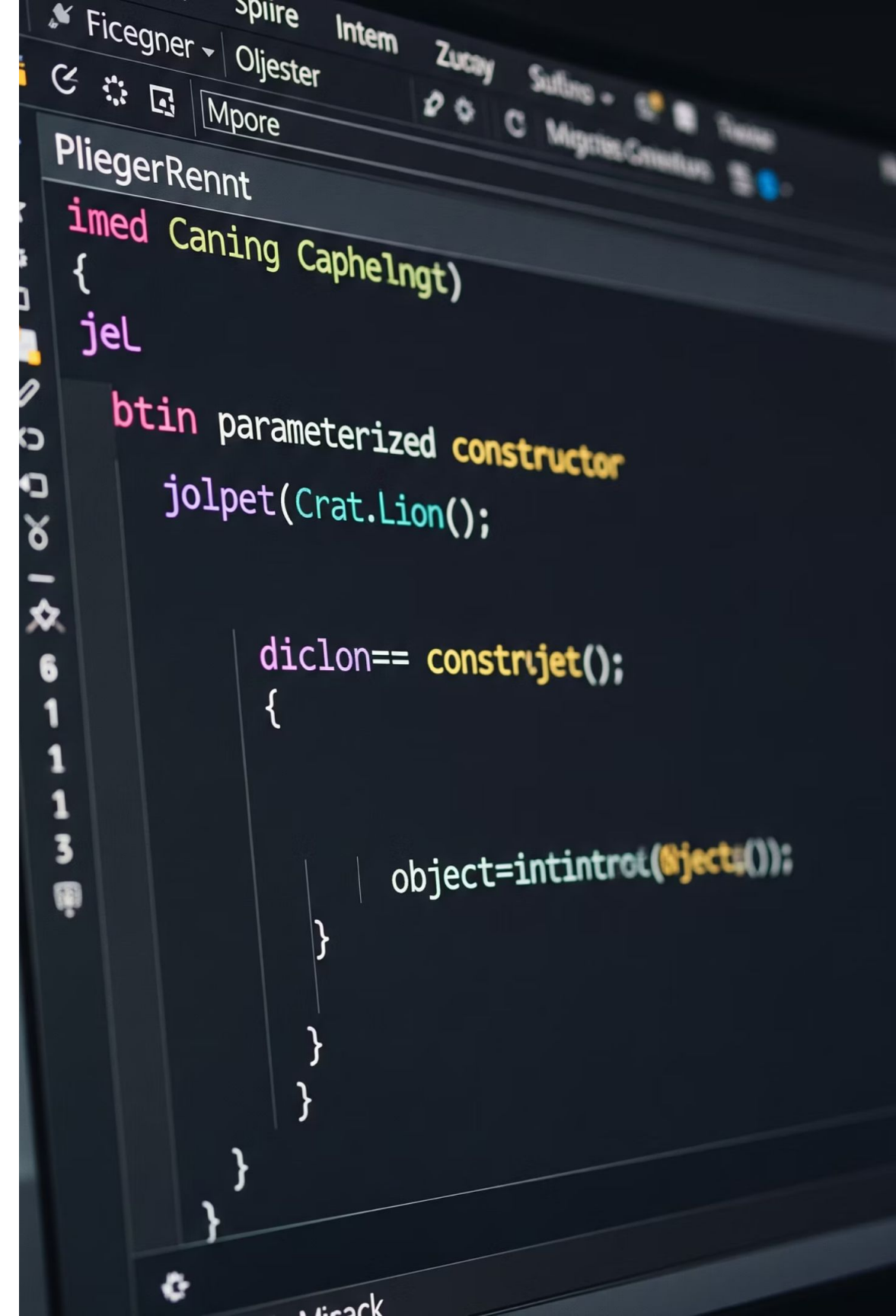
Exemplo Prático

```
public class Carro {  
    String modelo;  
    int ano;  
    double preco;  
  
    // Construtor padrão explícito  
    public Carro() {  
        this.modelo = "Indefinido";  
        this.ano = 2024;  
        this.preco = 0.0;  
    }  
}  
  
// Uso:  
Carro c = new Carro();  
System.out.println(c.modelo);  
// Saída: Indefinido
```

Construtor Parametrizado

O construtor parametrizado permite criar objetos com um **estado inicial definido**, recebendo argumentos no momento da instânciação com `new`. Isso garante que os dados essenciais sejam fornecidos desde o início, tornando o código mais robusto e expressivo.

```
Caneta.java
1
2 public class Caneta {
3
4     private String strCor;
5     private String strUso;
6     private boolean blnRetratil;
7     private String strModeloFabricante;
8
9     public Caneta() {
10
11     }
12
13     public Caneta(String strCor, String strUso,
14                   boolean blnRetratil, String strModeloFabricante) {
15
16         this.strCor = strCor;
17         this.strUso = strUso;
18         this.blnRetratil = blnRetratil;
19         this.strModeloFabricante = strModeloFabricante;
20     }
21 }
22
```



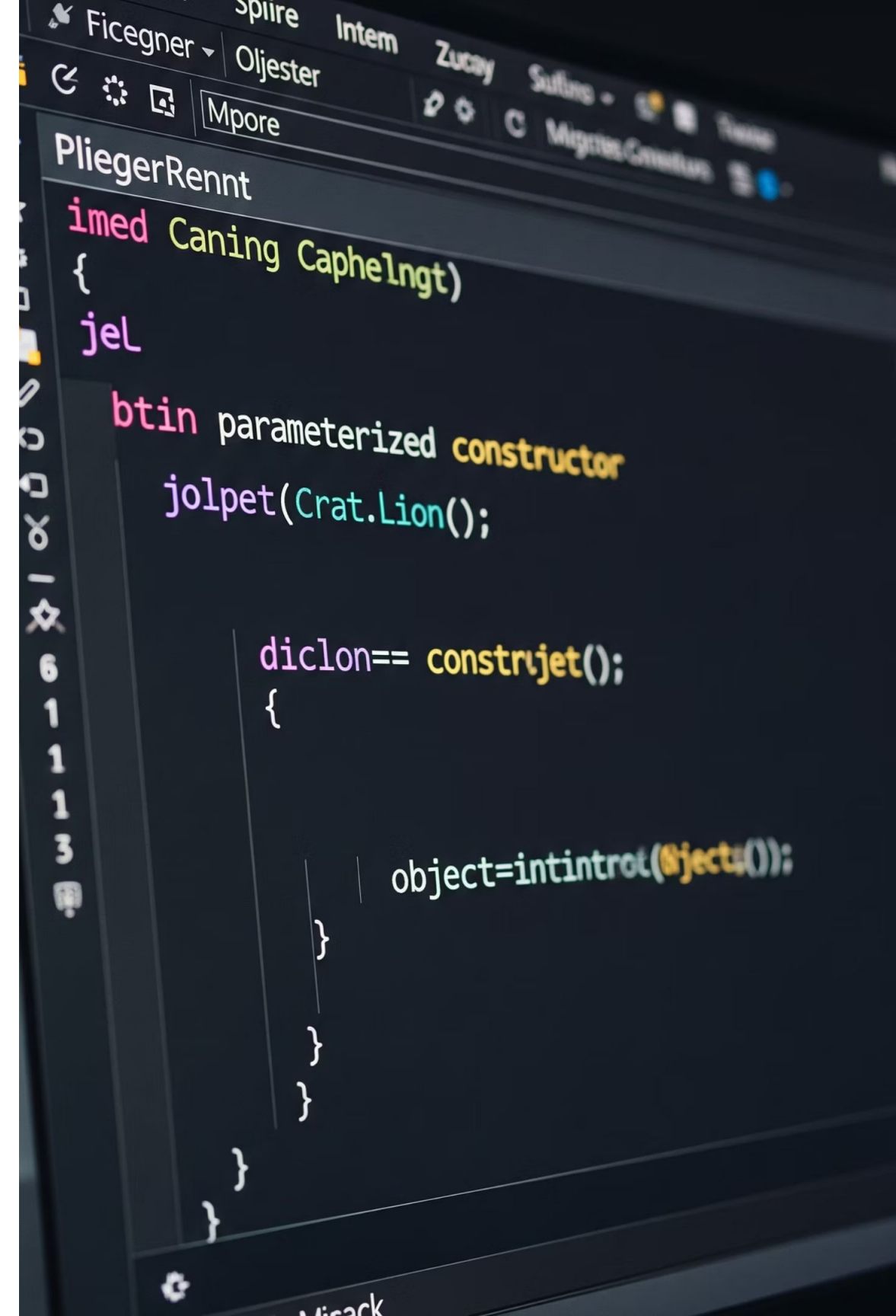
Construtor Parametrizado

Declaração

```
public class Pessoa {  
    String nome;  
    int idade;  
    String email;  
  
    public Pessoa(String nome,  
                   int idade,  
                   String email)  
{  
        this.nome = nome;  
        this.idade = idade;  
        this.email = email;  
    }  
}
```

Utilização

```
// Criando objetos  
Pessoa p1 = new Pessoa(  
    "Maria",  
    25,  
    "maria@email.com"  
);  
  
Pessoa p2 = new Pessoa(  
    "João",  
    30,  
    "joao@email.com"  
);  
  
System.out.println(p1.nome);  
// Saída: Maria
```



Exemplo Prático: Classe Aluno com Construtores

A classe `Aluno` abaixo demonstra a **sobrecarga de construtores**, múltiplos construtores com assinaturas diferentes coexistindo na mesma classe, oferecendo flexibilidade na criação de objetos.

Definição da Classe

```
public class Aluno {  
    String nome;  
    int matricula;  
    double nota;  
  
    // Construtor padrão  
    public Aluno() {  
        this.nome = "Sem nome";  
        this.matricula = 0;  
        this.nota = 0.0;  
    }  
}
```

```
// Construtor parametrizado  
public Aluno(String nome,int matricula) {  
    this.nome = nome;  
    this.matricula = matricula;  
    this.nota = 0.0;  
}  
  
// Construtor completo  
public Aluno(String nome, int matricula,double nota) {  
    this.nome = nome;  
    this.matricula = matricula;  
    this.nota = nota;  
}  
}
```

Exemplo Prático: Classe Aluno com Construtores

A classe `Aluno` abaixo demonstra a **sobrecarga de construtores**, múltiplos construtores com assinaturas diferentes coexistindo na mesma classe, oferecendo flexibilidade na criação de objetos.

Criando Objetos

```
// Usando construtor padrão
Aluno a1 = new Aluno();

// Usando construtor parcial
Aluno a2 = new Aluno(
    "Ana", 2024001
);

// Usando construtor completo
Aluno a3 = new Aluno(
    "Carlos", 2024002, 8.5
);
```

```
System.out.println(a1.nome);
// Saída: Sem nome
```

```
System.out.println(a2.nome);
// Saída: Ana
```

```
System.out.println(a3.nota);
// Saída: 8.5
```

▢ Cada chamada `new Aluno(...)` cria um objeto **independente** na memória heap da JVM.

A Palavra-chave `this`

A palavra-chave `this` é uma **referência ao próprio objeto** em execução. É indispensável dentro de construtores e métodos para diferenciar os **atributos da instância** dos **parâmetros** recebidos com o mesmo nome.

`this.atributo =` `parâmetro`

Disambigua atributo da classe do parâmetro local de mesmo nome.

`this()` — chama outro construtor

Invoca outro construtor da mesma classe. Deve ser a **primeira instrução** do construtor.

```
public class Produto {  
    String nome;  
    double preco;  
    int estoque;  
  
    public Produto(String nome,  
                    double preco,  
                    int estoque) {  
        // "this.nome" = atributo  
        // "nome" = parâmetro  
        this.nome = nome;  
        this.preco = preco;  
        this.estoque = estoque;  
    }  
  
    public void exibir() {  
        // "this" referencia o objeto  
        System.out.println(  
            "Produto: " + this.nome  
        );  
    }  
}
```

Chamando Construtores com `this()`

O uso de `this()` permite que um construtor **chame outro construtor** da mesma classe, evitando duplicação de código e centralizando a lógica de inicialização. Esta chamada **obrigatoriamente deve ser a primeira instrução** do construtor.

- ❏ Com `this()`, toda a lógica de inicialização fica concentrada em **um único construtor**, reduzindo erros e facilitando manutenção.

Com Duplicação (❌ Ruim)

```
public Veiculo() {
    this.marca = "Genérico";
    this.ano = 2024;
    this.cor = "Branco";
    // Código repetido!
}

public Veiculo(String marca) {
    this.marca = marca;
    this.ano = 2024;
    this.cor = "Branco";
    // Código repetido!
}
```

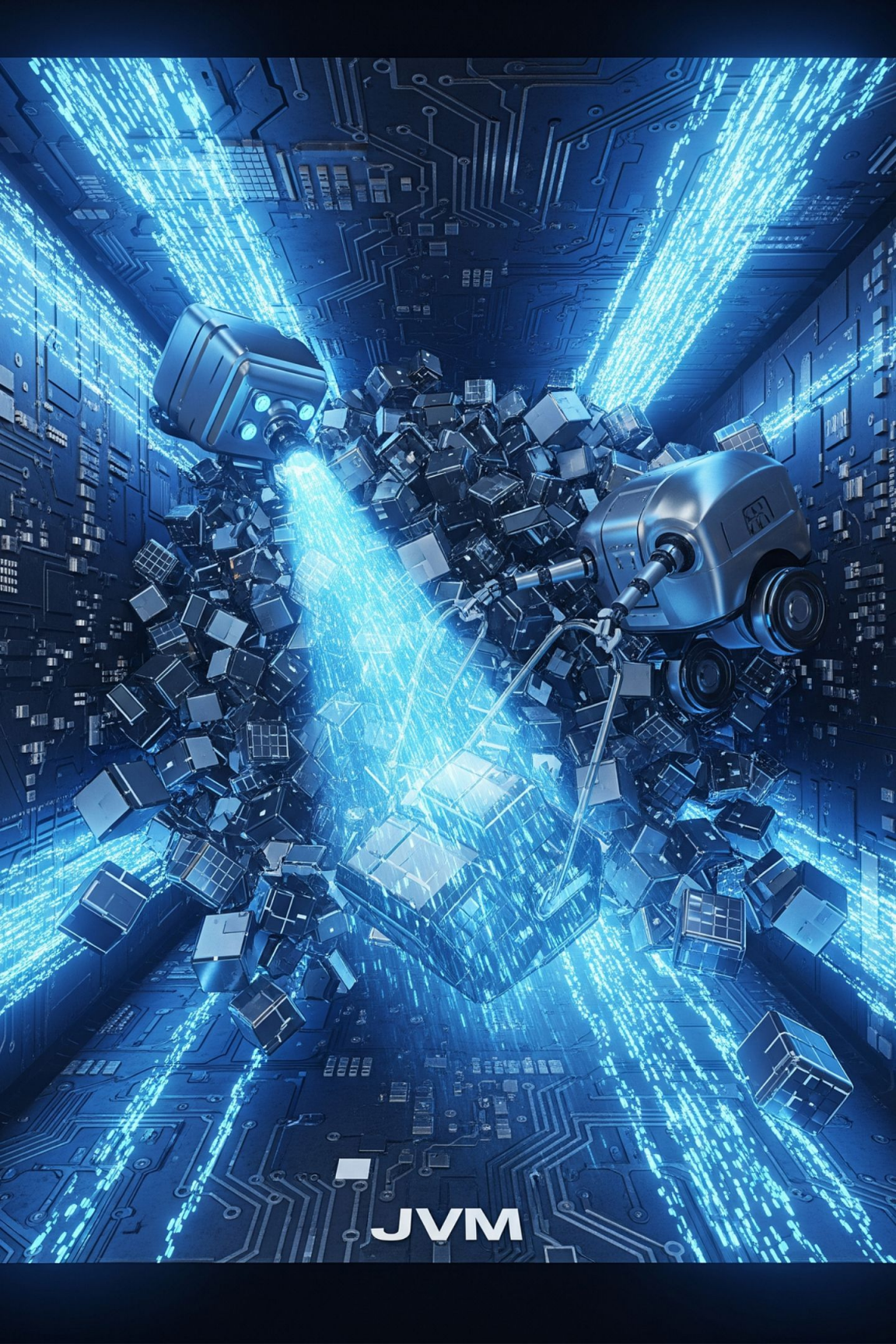

Chamando Construtores com `this()`

O uso de `this()` permite que um construtor **chame outro construtor** da mesma classe, evitando duplicação de código e centralizando a lógica de inicialização. Esta chamada **obrigatoriamente deve ser a primeira instrução** do construtor.

- ❏ Com `this()`, toda a lógica de inicialização fica concentrada em **um único construtor**, reduzindo erros e facilitando manutenção.

Com `this()` (✅ Bom)

```
public Veiculo() {  
    // Chama o construtor abaixo  
    this("Genérico", 2024, "Branco");  
}  
  
public Veiculo(String marca) {  
    // Chama o construtor completo  
    this(marca, 2024, "Branco");  
}  
  
public Veiculo(String marca,  
                int ano,  
                String cor) {  
    this.marca = marca;  
    this.ano = ano;  
    this.cor = cor;  
}
```



JVM

"Destructor" em Java - O Conceito

Diferente de linguagens como **C++**, Java **não possui destrutores explícitos**. O programador não controla diretamente quando ou como um objeto é destruído. Essa responsabilidade é delegada ao **Garbage Collector** da JVM, que gerencia automaticamente a liberação de memória.

Java (Automático)

A JVM detecta objetos sem referência e os remove automaticamente. O programador não precisa se preocupar com liberação de memória.



C++ (Explícito)

O programador deve declarar explicitamente o destrutor (`~NomeDaClasse()`) e chamar `delete` para liberar memória manualmente.

Garbage Collection — Gerenciamento Automático

Como Funciona

O **Garbage Collector (GC)** é um subsistema da JVM que monitora continuamente os objetos na memória heap. Quando um objeto não possui mais nenhuma referência ativa apontando para ele, ele se torna **elegível para coleta**.

- Remove objetos **sem referência** na memória heap
 - Executado pela JVM em **momentos apropriados**, de forma assíncrona
 - `System.gc()` apenas **sugere** a execução — não garante
- ❏ Nunca dependa de `System.gc()` em código de produção. O GC tem sua própria lógica otimizada.

Exemplo Prático

```
public class Exemplo {  
    public static void main(String[] args) {  
  
        // Objeto criado  
        Pessoa p = new Pessoa("Ana");  
  
        // p aponta para o objeto  
        System.out.println(p.nome);  
  
        // Removendo referência  
        p = null;  
  
        // Objeto elegível para GC  
        // (sem referência ativa)  
  
        // Sugerindo coleta (não garante)  
        System.gc();  
  
    }  
}
```

O Método `finalize()` e suas Alternativas

`finalize()` — Defasado ⚠

O método `finalize()` era chamado pela JVM **antes** de um objeto ser coletado pelo GC. Contudo, foi **deprecado no Java 9** e é considerado não confiável, pois:

- Não há garantia de quando será chamado
- Pode degradar o desempenho da aplicação
- Pode nunca ser executado em alguns cenários
- Dificulta o raciocínio sobre o ciclo de vida

Alternativa Moderna: `try-with-resources`

```
public class Conexao
    implements AutoCloseable {

    public Conexao() {
        System.out.println(
            "Conexão aberta"
        );
    }

    @Override
    public void close() {
        System.out.println("Conexão fechada");
    }
}

// Uso seguro:
try (Conexao c = new Conexao()) {
    // usa a conexão
} // close() chamado automaticamente
```

Ciclo de Vida de um Objeto em Java



Todo objeto Java passa por estas quatro etapas. A fase de **criação** envolve o construtor; a de **utilização** é o tempo de vida ativo; a de **inalcançável** ocorre quando não há mais variáveis referenciando o objeto; e a de **coleta** é quando o Garbage Collector recupera a memória ocupada.

Boas Práticas com Construtores

Inicialize Atributos Importantes

Garanta que todos os atributos essenciais recebam valores adequados no construtor, evitando `NullPointerException` durante o uso do objeto.

Use Sobrecarga com Propósito

Ofereça múltiplos construtores para flexibilizar a criação, mas combine-os com `this()` para centralizar a lógica e evitar duplicação.

Mantenha Construtores Simples

Evite lógica complexa, chamadas de rede ou operações de I/O dentro de construtores. Construtores devem apenas inicializar — nada mais.

Documente com JavaDoc

Utilize `/** ... */` para documentar cada construtor, descrevendo os parâmetros com `@param` e o comportamento esperado.



Exercícios de Fixação

PRÁTICA

1

Múltiplos Construtores

Crie a classe `Livro` com atributos `titulo`, `autor` e `paginas`. Implemente três construtores: padrão, com título apenas, e completo. Instancie objetos de cada forma.

2

Reutilização com `this()`

Refatore os construtores do exercício anterior usando `this()`, centralizando toda a lógica de inicialização no construtor completo. Verifique que o comportamento é idêntico.

3

Ciclo de Vida

Escreva um programa que crie 3 objetos `Aluno`, atribua `null` a dois deles, e chame `System.gc()`. Adicione mensagens de print para visualizar o ciclo. Implemente `AutoCloseable` como bônus.

4

Questões Teóricas

Responda: (a) O que acontece com o construtor padrão ao declararmos um construtor explícito? (b) Por que `finalize()` é deprecado? (c) Qual a diferença entre Java e C++ no gerenciamento de memória?



Na próxima aula, aprofundaremos o conceito de **sobrecarga de métodos** — um natural desdobramento do que aprendemos hoje com a sobrecarga de construtores.